

CSE488: Big Data Analytics

Lecture Notes — Vector Databases, LLMs, RAG, and FAISS Indexing

From Embeddings to Approximate Nearest Neighbor Search: kNN, IVF, PQ, IVF+PQ, HNSW

Summer 2026

0. Lecture Roadmap

This lecture connects three ideas you have already met in this course — nearest-neighbor search, clustering, and hashing (LSH) — to the modern problem of retrieving relevant text for Large Language Models (LLMs). By the end of this lecture you should be able to: (1) explain why raw brute-force k-NN does not scale to billion-vector corpora; (2) derive and hand-compute IVF, Product Quantization (PQ), IVF+PQ, and HNSW on small worked examples; (3) read and write FAISS Python code for each index type; and (4) describe how a Retrieval-Augmented Generation (RAG) pipeline is built on top of these indexes.

1. From Text to Vectors: Embeddings

An embedding model f maps an object x (a word, sentence, document, or image) into a dense real-valued vector $f(x)$ in $\mathbb{R}^d$, such that objects with similar meaning end up close together under some distance function. Modern LLM-based encoders (e.g., BERT-style or LLM embedding APIs) typically produce $d = 384, 768$, or 1536 -dimensional vectors.

Formally, an embedding function is:

$$f: X \rightarrow \mathbb{R}^d, \quad f(\text{"the cat sat on the mat"}) = [0.021, -0.183, 0.402, \dots, 0.077]$$

The key design goal is that semantic similarity in the real world should correspond to geometric proximity in $\mathbb{R}^d$. This is what allows us to replace "does this document answer the query?" — a hard language problem — with "which vectors are closest to the query vector?" — a geometric problem we know how to solve efficiently.

1.1 Similarity / Distance Metrics

Metric	Formula	Notes
Euclidean (L2)	$d(x,y) = \sqrt{\sum_i (x_i - y_i)^2}$	Smaller = more similar; sensitive to vector magnitude
Cosine similarity	$\cos(x,y) = (x \cdot y) / (\ x\ \ y\)$	Ignores magnitude, measures angle only; range $[-1,1]$

Metric	Formula	Notes
Dot product (inner product)	$x \cdot y = \sum_i x_i y_i$	Equivalent to cosine if vectors are L2-normalized

1.2 Worked Numerical Example

Let $x = (1, 2, 2)$ and $y = (2, 2, 1)$ be two 3-dimensional embeddings (toy dimension, for hand computation).

```
L2 distance:
d(x,y) = sqrt( (1-2)^2 + (2-2)^2 + (2-1)^2 )
        = sqrt( 1 + 0 + 1 ) = sqrt(2) ~ 1.414

Dot product:
x . y = (1)(2) + (2)(2) + (2)(1) = 2 + 4 + 2 = 8

Norms:
||x|| = sqrt(1^2+2^2+2^2) = sqrt(9) = 3
||y|| = sqrt(2^2+2^2+1^2) = sqrt(9) = 3

Cosine similarity:
cos(x,y) = 8 / (3 * 3) = 8/9 ~ 0.889
```

A cosine similarity of 0.889 (close to 1) tells us x and y point in nearly the same direction — they would be considered semantically close even though their L2 distance is not zero.

1.3 Python: computing embeddings similarity

```
import numpy as np

x = np.array([1, 2, 2], dtype=float)
y = np.array([2, 2, 1], dtype=float)

l2 = np.linalg.norm(x - y)
cos_sim = np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))

print(f'L2 distance    = {l2:.3f}')
print(f'Cosine sim     = {cos_sim:.3f}')
# L2 distance    = 1.414
# Cosine sim     = 0.889
```

2. Exact k-Nearest-Neighbor (k-NN) Search

Given a query vector q and a database of N vectors $\{v_1, \dots, v_N\}$ in $\mathbb{R}^d$, exact k-NN search finds the k vectors with the smallest distance (or highest similarity) to q by computing the distance to every single vector and sorting.

2.1 Complexity

Brute-force k-NN costs $O(N \times d)$ to compute all distances, plus $O(N \log k)$ (or $O(N)$ with a heap of size k) to select the top- k . For a corpus of $N = 10^9$ vectors at $d = 768$, a single query requires $\sim 7.68 \times 10^{11}$ floating-point multiply-adds. Even at modern hardware throughput, this is far too slow for interactive, low-latency retrieval when there are many queries per second — this is exactly the motivation for Approximate Nearest Neighbor (ANN) search.

2.2 Worked Example: brute-force k-NN by hand

Database of 6 two-dimensional vectors and query $q = (5, 4)$:

id	vector	L2 distance to $q=(5,4)$	rank
v1	(1, 2)	$\sqrt{16+4}=4.47$	5
v2	(6, 5)	$\sqrt{1+1}=1.41$	1
v3	(2, 8)	$\sqrt{9+16}=5.00$	6
v4	(7, 3)	$\sqrt{4+1}=2.24$	3
v5	(5, 6)	$\sqrt{0+4}=2.00$	2
v6	(4, 1)	$\sqrt{1+9}=3.16$	4

For $k = 2$, exact k-NN returns $\{v2, v5\}$ — the two closest points. This is trivial at $N = 6$, but note that we touched every single vector to be certain of correctness; that exhaustive touch is precisely what ANN methods try to avoid.

2.3 Python: brute-force k-NN

```
import numpy as np

db = np.array([[1,2],[6,5],[2,8],[7,3],[5,6],[4,1]], dtype=float)
q = np.array([5,4], dtype=float)

dists = np.linalg.norm(db - q, axis=1)
k = 2
nn_idx = np.argsort(dists)[:k]

print('distances:', dists)
print('top-k ids:', nn_idx, 'distances:', dists[nn_idx])
# top-k ids : [1 4]  distances: [1.41 2.0]    (v2, v5)
```

Bridge to FAISS

FAISS calls brute-force exact search “IndexFlatL2” (L2) or “IndexFlatIP” (inner product). It is exact and simple, and it is the correct baseline to compute recall against when you evaluate any approximate index below.

3. Why Approximate Nearest Neighbor (ANN) Search?

Real vector databases used behind LLM applications hold from millions to billions of embeddings (e.g., a company's entire document corpus, chunked into passages). At that scale, brute-force search is too slow, and — just as importantly — every vector must be kept in RAM in float32, which becomes expensive: **1 billion vectors at $d = 768$ dimensions and 4 bytes/float requires $1,000,000,000 \times 768 \times 4$ bytes ≈ 2.86 TB of memory just for the raw vectors.**

ANN methods trade a small, controllable amount of accuracy (recall) for large gains in speed and/or memory. FAISS (Facebook AI Similarity Search) is the most widely used open-source library implementing these trade-offs. We will study four core FAISS index families:

- IVF (Inverted File Index) — reduces the search space by clustering.

- PQ (Product Quantization) — reduces memory by compressing vectors.
- IVF+PQ — combines both: search fewer vectors, and each is compressed.
- HNSW (Hierarchical Navigable Small World) — a graph-based method with excellent speed/recall trade-off, at higher memory cost.

4. IVF: Inverted File Index

IVF is a coarse-quantization method. The core idea, directly building on the k-means clustering you learned earlier this semester, is: partition the vector space into $nlist$ clusters (Voronoi cells) using k-means; build an inverted list for each cluster containing the ids of every database vector assigned to it; at query time, instead of scanning all N vectors, only scan the vectors inside the $nprobe$ cluster(s) whose centroid is closest to the query.

4.1 Index-Time (Build) Algorithm

1. Run k-means on a (sample of the) database to obtain $nlist$ centroids $c_1, \dots, c_{nlist}$.
2. For every database vector v_i , assign it to its nearest centroid: $cell(v_i) = \text{argmin}_j \|v_i - c_j\|$.
3. Store an inverted list: $list[j] = \{i : cell(v_i) = j\}$.

4.2 Query-Time (Search) Algorithm

1. Compute distance from query q to all $nlist$ centroids.
2. Select the $nprobe$ closest centroids.
3. Scan only the vectors in those $nprobe$ inverted lists, computing exact distance to q .
4. Return the top- k of the scanned vectors.

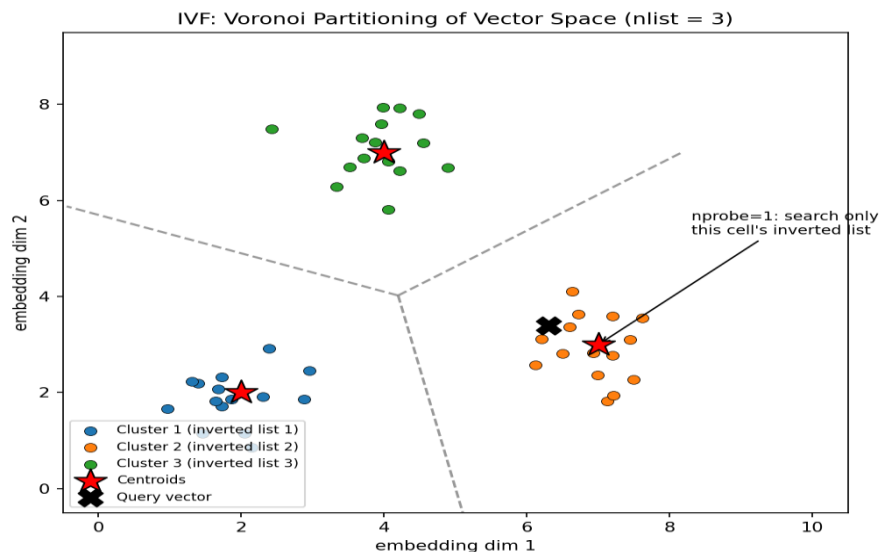


Figure 1. IVF partitions the embedding space into Voronoi cells around k-means centroids. With $nprobe=1$, a query only searches the inverted list of the single nearest cell — here, Cluster 2.

4.3 Worked Numerical Example

Database of 8 points in $\mathbb{R}^2$, clustered into $nlist = 2$ clusters via k-means:

id	vector	assigned cluster
v1	(1, 1)	A
v2	(1, 2)	A
v3	(2, 1)	A
v4	(2, 3)	A
v5	(8, 8)	B
v6	(9, 7)	B
v7	(7, 9)	B
v8	(8, 9)	B

Centroid A = mean of {v1..v4} = (1.5, 1.75). Centroid B = mean of {v5..v8} = (8.0, 8.25).

Query $q = (1.8, 1.6)$. Distance to centroid A = $\text{sqrt}((1.8-1.5)^2 + (1.6-1.75)^2) = \text{sqrt}(0.09 + 0.0225) = 0.335$.
Distance to centroid B = $\text{sqrt}((1.8-8)^2 + (1.6-8.25)^2) \approx 10.13$.

With $n_{\text{probe}} = 1$, IVF searches only cluster A's 4 vectors instead of all 8 — a 2x reduction here, but the reduction grows to orders of magnitude when n_{list} is in the thousands, and the database has billions of vectors.

4.4 The n_{probe} / recall / speed trade-off

n_{probe} controls how many clusters are scanned. $n_{\text{probe}} = n_{\text{list}}$ reduces IVF to exact brute-force search (100% recall, no speedup). Small n_{probe} is fast but risks missing true neighbors that fall just across a cell boundary — this is the classic "boundary problem" of any partition-based ANN method. In practice, n_{probe} is tuned on a validation set to hit a target recall (e.g., $\text{recall}@10 \geq 0.95$).

4.5 FAISS Python: IndexIVFFlat

```
import faiss
import numpy as np

d = 128                # embedding dimension
nlist = 100            # number of Voronoi cells

quantizer = faiss.IndexFlatL2(d)          # used to assign vectors to clusters
index = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_L2)

xb = np.random.random((100000, d)).astype('float32') # database
index.train(xb)          # runs k-means to find the nlist centroids
index.add(xb)            # assigns each vector to its inverted list

index.nprobe = 8         # search 8 of the 100 clusters
xq = np.random.random((5, d)).astype('float32')
D, I = index.search(xq, k=10) # D = distances, I = neighbor ids
```

Key requirement

IndexIVFFlat needs `index.train()` to be called before `index.add()` — training is where k-means learns the centroids. This differs from `IndexFlatL2`, which has no training step at all.

5. Product Quantization (PQ)

IVF reduces how many vectors we compare against, but each vector is still stored and compared in full float32 precision — expensive in memory. Product Quantization instead compresses each vector itself, trading a controlled amount of precision for a dramatic reduction in memory footprint, while still allowing fast approximate distance computation.

5.1 The Idea

1. Split every D-dimensional vector into m sub-vectors, each of dimension D/m.
2. For each of the m sub-vector "slots", independently run k-means (with k* centroids, typically k* = 256 so each id fits in 1 byte) on the corresponding sub-vectors across the whole database. This produces m codebooks.
3. Encode each database vector by replacing every sub-vector with the id of its nearest codeword in that slot's codebook. A D-dimensional float32 vector (4D bytes) becomes just m bytes.

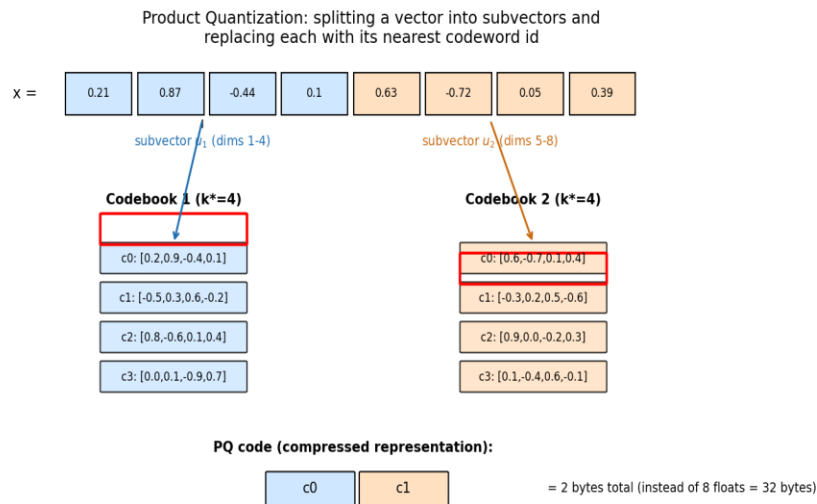


Figure 2. An 8-dimensional vector is split into 2 subvectors of dimension 4. Each subvector is replaced by the id of its nearest codeword in its own codebook ($k^*=4$ here for illustration; in practice $k^*=256$). The vector is now stored as 2 bytes instead of 32 bytes.

5.2 Worked Numerical Example

Let $D = 4$, split into $m = 2$ sub-vectors of dimension 2. Suppose codebook 1 (for dims 1-2) has 2 codewords, and codebook 2 (for dims 3-4) has 2 codewords:

Codebook 1	value	Codebook 2	value
c1_0	(0, 0)	c2_0	(5, 5)
c1_1	(3, 3)	c2_1	(1, 1)

Vector to encode: $x = (2.9, 3.1, 1.2, 0.8)$. Split: $u_1 = (2.9, 3.1)$, $u_2 = (1.2, 0.8)$.

```
Distance u1 to c1_0=(0,0): sqrt(2.9^2+3.1^2) = sqrt(8.41+9.61)=4.24
Distance u1 to c1_1=(3,3): sqrt(0.1^2+0.1^2) = sqrt(0.02)=0.14    <- nearest
=> u1 encoded as c1_1

Distance u2 to c2_0=(5,5): sqrt(3.8^2+4.2^2) = 5.66
```

```
Distance u2 to c2_1=(1,1): sqrt(0.2^2+0.2^2) = 0.28 <- nearest
=> u2 encoded as c2_1

PQ code for x = (c1_1, c2_1) -- just 2 small integers instead of 4 floats
Reconstructed x_hat = (3,3,1,1) (concatenation of the chosen codewords)
```

5.3 Distance Computation: Asymmetric Distance Computation (ADC)

At query time, we do NOT decode database vectors back to float32. Instead, for a query q , we split q into the same m sub-vectors and precompute a small lookup table per slot: the distance from q 's sub-vector to every one of the k^* codewords in that slot's codebook. The approximate distance between q and a compressed database vector is then just a sum of m table look-ups — no floating point distance computation against the reconstructed vector is needed:

$$d_{PQ}(q, x)^2 \approx \sum_{j=1}^m \|q_j - \text{codebook}_j[\text{code}_j(x)]\|^2$$

This lookup-table trick is what makes PQ distance computation extremely fast even though the stored vectors are heavily compressed.

The algorithm, step by step

Step 1 — Split the query. Just like every database vector, split q into m subvectors: $q \rightarrow (q_1, q_2, \dots, q_m)$, each of dimension D/m .

Step 2 — Build a distance table (once per query). For each subquantizer $j = 1..m$, compute the squared distance from q_j to *every one* of the k^* codewords in codebook j :

$\text{Table}[j][k] = \|q_j - \text{codebook}_j[k]\|^2$ for $k = 0, \dots, k^*-1$

This gives m small tables of size k^* each — for the standard $k^* = 256$, $m = 16$ setting, that's $16 \times 256 = 4096$ precomputed numbers, done **once per query**, not once per database vector.

Step 3 — Look up and sum, per database vector. For a database vector x with PQ code $(\text{code}_1(x), \dots, \text{code}_m(x))$, its approximate squared distance to q is just:

$$d_{PQ}(q, x)^2 \approx \sum_j \text{Table}[j][\text{code}_j(x)]$$

No floating-point distance computation happens here at all — just m array look-ups and m additions, per database vector.

Worked numerical example

Reuse the lecture's codebooks ($D=4$, $m=2$, $k^*=2$):

Codebook 1 (dims 1–2) value Codebook 2 (dims 3–4) value

$c1_0$ $(0, 0)$ $c2_0$ $(5, 5)$

$c1_1$ $(3, 3)$ $c2_1$ $(1, 1)$

Database vector $x = (2.9, 3.1, 1.2, 0.8) \rightarrow$ PQ code $(1, 1)$ — as computed in the lecture.

Now a new query: $q = (2.5, 3.5, 0.9, 1.3)$, split into $q_1=(2.5, 3.5)$, $q_2=(0.9, 1.3)$.

Step 1 — build the distance table:

Compute ADC distance tables

Compute ADC distance tables

This gives us the full lookup tables:

Table 1 = [18.5, 0.5] **Table 2** = [30.5, 0.10]

Step 2 — answer the query for x with code (1,1):

$d_{PQ}(q, x)^2 \approx \text{Table1}[1] + \text{Table2}[1] = 0.5 + 0.10 = \mathbf{0.60}$

Let's check this against the *true* squared L2 distance (as if x had never been compressed):

Compute true distances for comparison

Compute true distances for comparison

Now let's check a second, clearly distant database vector $y = (0.1, 0.2, 4.8, 5.1)$, which encodes to PQ code (0, 0):

$d_{PQ}(q, y)^2 \approx \text{Table1}[0] + \text{Table2}[0] = 18.5 + 30.5 = \mathbf{49.0}$ (true value: 46.3)

Vector	PQ-approx d ²	True d ²	Correctly ranked?
x (code 1,1)	0.60	0.66	✓ nearer
y (code 0,0)	49.0	46.3	✓ farther

The absolute numbers are slightly off (that's the quantization error — x and y aren't exactly at their codewords), but **the ranking is preserved**, which is all a top-k search actually needs. This is the core promise of ADC: individual distances are approximate, but relative ordering is usually reliable enough for high-recall retrieval.

5.4 Memory savings

Representation	Bytes per vector (D=768)	1 billion vectors
Raw float32	768 x 4 = 3072 bytes	~2.86 TB
PQ (m=96, 1 byte/code)	96 bytes	~89 GB
PQ (m=48, 1 byte/code)	48 bytes	~45 GB

5.5 FAISS Python: IndexPQ

```
import faiss
import numpy as np

d = 128          # vector dimension (must be divisible by m)
m = 16           # number of subvectors
nbits = 8        # bits per code -> k* = 2^nbits = 256 codewords per codebook

index = faiss.IndexPQ(d, m, nbits)

xb = np.random.random((100000, d)).astype('float32')
index.train(xb)    # learns m codebooks, each via k-means with k*=256
index.add(xb)      # stores only the PQ codes (m bytes per vector)
```

```
xq = np.random.random((5, d)).astype('float32')
D, I = index.search(xq, k=10)
```

6. Combining IVF and PQ

IVF alone reduces how many vectors are scanned, but stores full-precision vectors in each inverted list. PQ alone compresses every vector but still requires scanning the entire database. IVF+PQ combines both: IVF narrows the search to a handful of clusters, and within those clusters, vectors are stored and compared using compact PQ codes. This is the workhorse configuration for very large production vector search systems.

6.1 Residual Encoding

A key refinement is that IVF+PQ does not PQ-encode the raw vector — it PQ-encodes the residual after subtracting the assigned IVF centroid:

$r(v) = v - c_{\{cell(v)\}}$ then PQ-encode $r(v)$ instead of v

Intuitively, all vectors inside one Voronoi cell already share most of their "location" information (the centroid); encoding only the residual concentrates the PQ codebooks' capacity on the fine-grained differences that actually distinguish vectors within a cell, which improves accuracy for the same code length.

6.2 Combined Build & Search Pipeline

Stage	IVF role	PQ role
Build	k-means -> nlist centroids; assign each vector to a cell	k-means per subvector -> m codebooks on residuals
Store	inverted list of ids per cell	m-byte PQ code per vector (of the residual)
Search	find nprobe nearest cells to q	ADC lookup-table distance within those cells only

6.3 Worked Example: putting it together

Continuing Section 4.3's example (nlist = 2, centroid A = (1.5, 1.75)): vector $v_2 = (1, 2)$ belongs to cell A. Its residual is $r(v_2) = (1, 2) - (1.5, 1.75) = (-0.5, 0.25)$. This small residual (rather than the original (1,2)) is what gets PQ-encoded — residuals are typically much smaller in magnitude and more uniformly distributed, which PQ quantizes more accurately.

6.4 FAISS Python: IndexIVFPQ

```
import faiss
import numpy as np

d = 128
nlist = 100      # IVF: number of clusters
m = 16           # PQ: number of subvectors (must divide d)
nbits = 8        # PQ: bits per code

quantizer = faiss.IndexFlatL2(d)
index = faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits)

xb = np.random.random((1000000, d)).astype('float32')
index.train(xb)   # learns BOTH the nlist centroids AND the m codebooks
```

```

index.add(xb)

index.nprobe = 10
xq = np.random.random((5, d)).astype('float32')
D, I = index.search(xq, k=10)

# Memory per vector here: only m=16 bytes (+ small per-list overhead),
# vs. 128*4=512 bytes for the raw float32 vector -> 32x compression.

```

Why this matters at scale

IndexIVFPQ is the classic choice for datasets that do not fit in RAM as raw float32. It is the default recommendation in the FAISS documentation once a dataset passes roughly 1 million vectors.

7. HNSW: Hierarchical Navigable Small World Graphs

HNSW takes a completely different approach from IVF/PQ: instead of partitioning or compressing the space, it builds a multi-layer proximity graph over the database vectors and answers queries with greedy graph traversal. It currently offers one of the best speed/recall trade-offs of any ANN method, at the cost of higher memory use (it stores full vectors plus graph edges) and slower index construction.

7.1 Structure

HNSW builds several stacked layers of graphs, inspired by skip lists:

- Layer 0 (bottom): contains every vector in the database; densely connected to close neighbors.
- Higher layers: contain a shrinking, randomly-sampled subset of vectors, with progressively longer-range edges. Each vector is inserted into layer 0, and independently into layer 1, 2, ... with exponentially decaying probability, so the top layer has very few "hub" nodes.
- Each node in each layer keeps at most M edges to its approximate nearest neighbors in that layer (M is a build-time parameter, typically 16-64).

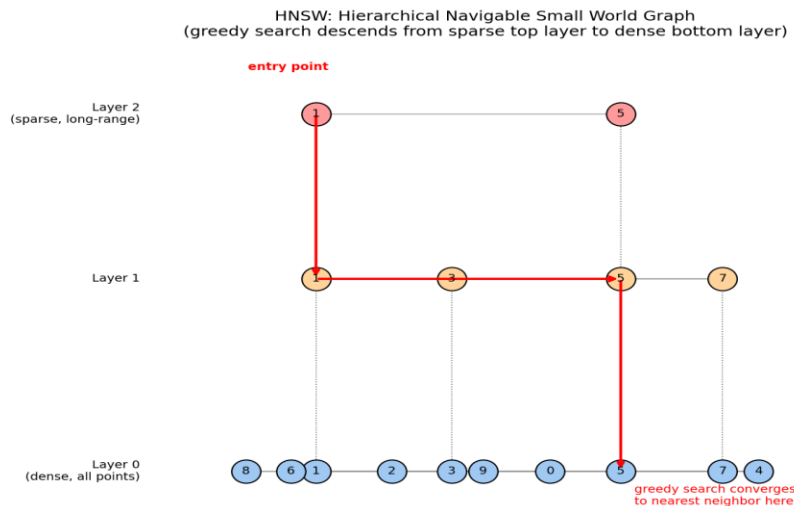


Figure 3. HNSW's layered graph. Search starts at an entry point in the sparse top layer and greedily moves to the closest neighbor at each step; upon reaching a local optimum, the search drops down one layer and continues, ending with a fine-grained search in the dense bottom layer.

7.2 Search Algorithm (Greedy Descent)

1. Start at a fixed entry point in the topmost layer.
2. At the current layer, greedily move to the neighbor (among the current node's graph edges) that is closest to the query q , repeating until no neighbor improves the distance (a local minimum is reached).
3. Drop down one layer, using the current best node as the new starting point, and repeat step 2.
4. At layer 0, perform a wider beam search (beam width $efSearch$) to collect the final top-k candidates rather than a single greedy point, to improve recall.

7.3 Why it is fast: skip-list intuition

This is directly analogous to a skip list's $O(\log N)$ search: the sparse top layers let the search take big "jumps" across the space quickly, roughly halving (or more) the remaining search radius at each layer, and only the final layer performs a careful local search. This gives HNSW roughly $O(\log N)$ query time in practice, compared to $O(N)$ for brute-force.

7.4 Key Parameters

Parameter	Effect
M	Max edges per node per layer. Higher M -> better recall & more memory (graph edges), slower build.
efConstruction	Beam width used while building the graph. Higher -> better graph quality, slower build.
efSearch	Beam width used while querying. Higher -> better recall, slower query. Tunable at query time without rebuilding.

7.5 FAISS Python: IndexHNSWFlat

```
import faiss
import numpy as np

d = 128
M = 32                                # edges per node per layer

index = faiss.IndexHNSWFlat(d, M)
index.hnsw.efConstruction = 200      # build-time beam width

xb = np.random.random((100000, d)).astype('float32')
index.add(xb)                        # no separate train() call needed

index.hnsw.efSearch = 64             # query-time beam width (tune per query load)
xq = np.random.random((5, d)).astype('float32')
D, I = index.search(xq, k=10)
```

HNSW vs IVF+PQ

HNSW generally gives higher recall at a given query latency, but stores full vectors + graph edges, using far more memory per vector than IVF+PQ. In production, HNSW is often paired with PQ-compressed vectors (FAISS's IndexHNSWPQ) to get graph-search speed with compressed memory.

8. Comparing the Four Methods

Method	Search complexity	Memory / vector	Build cost	Best when...
Flat (exact kNN)	$O(N)$	4D bytes (full)	none	small N , need exact / ground truth
IVF	$O(n_{\text{probe}}/n_{\text{list}} \times N)$	4D bytes (full)	k-means (moderate)	medium-large N , some memory available
PQ	$O(N)$ but cheap per-op	m bytes (tiny)	$m \times$ k-means	memory-bound, N not huge
IVF+PQ	$O(n_{\text{probe}}/n_{\text{list}} \times N)$, cheap per-op	m bytes (tiny)	k-means + $m \times$ k-means	very large N and memory-bound (production default)
HNSW	$\sim O(\log N)$	4D + graph edges (large)	graph construction (slow)	low-latency, recall-critical, memory available

A practical rule of thumb taught in the FAISS documentation: start with IndexFlatL2 to get a correctness baseline on a small sample; move to IndexIVFFlat or IndexHNSWFlat once N exceeds $\sim 10^5$ - 10^6 ; move to IndexIVFPQ once memory, not just speed, becomes the bottleneck (typically $N > 10^6$ - 10^7 with limited RAM).

9. LLM and Embedding Recap

A Large Language Model (LLM) is a transformer-based neural network trained to predict the next token in a sequence over massive text corpora. Two of its capabilities matter directly for this lecture:

- Generation: given a prompt, the LLM autoregressively generates fluent, contextually relevant text.
- Embedding: an LLM (or a smaller encoder trained the same way) can also be used purely to map text into the dense vector space discussed in Section 1 — this is the "embedding model" step of the RAG pipeline below.

A critical limitation of LLMs is that their knowledge is frozen at training time and limited to what fits in their finite context window; they cannot natively access private, proprietary, or newly published documents, and they can hallucinate — generate fluent but factually incorrect text — when asked about facts outside their training distribution. Retrieval-Augmented Generation directly targets this limitation.

10. Retrieval-Augmented Generation (RAG)

RAG combines a retrieval system (built on exactly the ANN indexes studied above) with a generative LLM, so that the model's answer is grounded in retrieved evidence rather than relying purely on parametric (trained-in) knowledge.

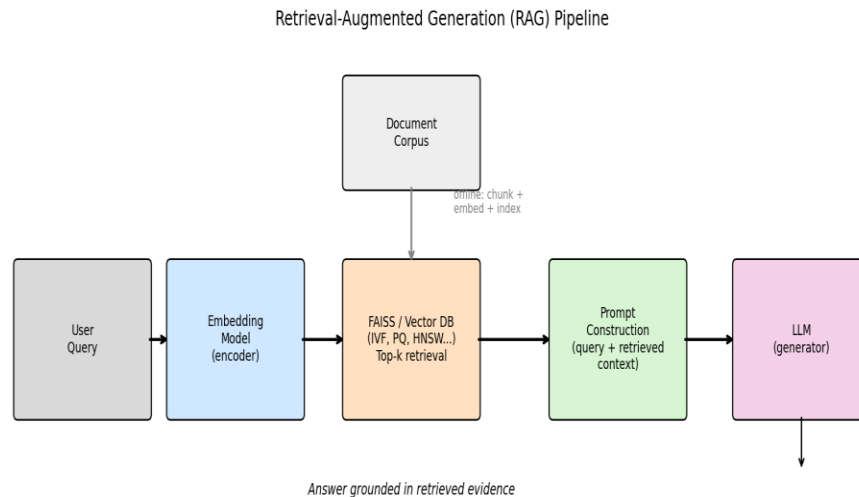


Figure 4. The RAG pipeline. Offline, a document corpus is chunked, embedded, and indexed (FAISS/vector DB). Online, a user query is embedded, the vector index returns the top-k most relevant chunks, and those chunks are inserted into the LLM's prompt as context before generation.

10.1 Pipeline Stages

5. Offline indexing: split documents into chunks (e.g., 200-500 tokens with overlap); embed every chunk with the embedding model; insert all chunk vectors into a FAISS/vector-database index (IVF, PQ, IVF+PQ, or HNSW depending on scale).
6. Query embedding: embed the user's query with the same embedding model, producing q in $\mathbb{R}^d$.
7. Retrieval: run ANN search (Sections 4-7) to find the top-k chunks most similar to q .
8. Augmentation: construct a prompt that concatenates the retrieved chunks as context, followed by the original query and an instruction.
9. Generation: the LLM produces an answer conditioned on both its own parameters and the retrieved context, ideally citing or grounding claims in the supplied chunks.

10.2 Why the ANN choice matters here

Retrieval quality directly bounds generation quality: if the top-k retrieved chunks do not contain the answer, no amount of LLM capability can recover it. This is exactly why the recall/speed/memory trade-offs of Sections 4-8 are not just a database-systems detail — they are a first-order factor in whether a RAG system actually answers questions correctly, and at what latency and cost.

10.3 Minimal Python RAG example using FAISS

```

import faiss
import numpy as np

# --- offline: build the index over document chunk embeddings ---
d = 384
chunk_embeddings = np.random.random((50000, d)).astype('float32') # from an embedding
model

index = faiss.IndexHNSWFlat(d, 32) # good default for moderate-size, low-latency RAG
index.add(chunk_embeddings)

# --- online: answer a query ---

```

```

query_embedding = np.random.random((1, d)).astype('float32') # embed(user_query)
D, I = index.search(query_embedding, k=5) # top-5 relevant chunk ids

retrieved_chunks = [chunk_texts[i] for i in I[0]] # look up original text by id
prompt = 'Context:\n' + '\n'.join(retrieved_chunks) + f'\n\nQuestion: {user_query}'
# prompt is then sent to the LLM for generation

```

10.4 Vector Databases vs. FAISS

FAISS is a library, not a full database: it provides fast in-process indexing and search but no built-in persistence layer, metadata filtering, or multi-user access control out of the box. Managed vector databases (e.g., Pinecone, Weaviate, Milvus, or the pgvector extension for PostgreSQL) typically use FAISS-style algorithms (IVF, PQ, HNSW) internally, while adding: durable storage, horizontal scaling/sharding across machines, metadata filtering (e.g., "only search chunks tagged year=2026"), hybrid search (combining vector similarity with keyword/BM25 search), and API/access-control layers for multi-tenant applications.

10.5 Connecting back to the course

Note the parallel to Locality-Sensitive Hashing (LSH), covered earlier in this course for approximate similarity search over sets (e.g., MinHashing document similarity): both LSH and FAISS's ANN indexes exist to avoid exhaustive $O(N)$ comparison, by pre-organizing data so that only a small, likely-relevant subset needs to be checked at query time. LSH uses hash buckets; IVF uses Voronoi cells; HNSW uses graph proximity — three different data structures solving the same underlying approximate-search problem.

11. Summary

- Embeddings turn text into vectors in R^d so that semantic similarity becomes geometric proximity, measured by L2 distance, cosine similarity, or dot product.
- Exact k-NN is correct but $O(N)$ per query — too slow and memory-hungry at billion-vector scale.
- IVF speeds up search by clustering the space (k-means) and only scanning the nprobe closest clusters' inverted lists.
- PQ shrinks memory by independently quantizing sub-vectors into small codes, using precomputed lookup tables (ADC) for fast approximate distance.
- IVF+PQ combines both: cluster to narrow the search, then compress residuals within each cluster — the standard production configuration for very large corpora.
- HNSW builds a multi-layer navigable graph for near $O(\log N)$ search with strong recall, at the cost of higher memory.
- RAG pipelines wrap an embedding model + a FAISS/vector-database ANN index (any of the above) + an LLM, so that generation is grounded in retrieved, up-to-date, verifiable evidence rather than only the model's frozen training data.

Suggested Exercises for Students

10. By hand, compute the IVF assignment and residual-encoded PQ code for a new query vector added to the Section 6.3 example.
11. Implement IndexFlatL2, IndexIVFFlat, and IndexHNSWFlat on the same dataset (e.g., 100k random vectors) and empirically compare query latency and recall@10 against the exact baseline.
12. Given a small text corpus, build a minimal RAG pipeline: chunk, embed (any sentence-transformers model), index with FAISS, and answer 3 sample questions, reporting which chunks were retrieved.